

Lesson 7

Material

<https://github.com/unixfile/learning-python-2026>

Today

- ▶ Objects and classes
- ▶ Initialisation and `self`
- ▶ Class attributes vs instance attributes
- ▶ Instance and static methods
- ▶ Magic methods
- ▶ Protected and private attributes
- ▶ Properties
- ▶ Data classes
- ▶ Inheritance
- ▶ Composition

1 Objects and classes

- ▶ An object bundles data (**attributes**) and behaviour (**methods**)
- ▶ Everything in Python is an object: integers, strings, lists, functions
- ▶ Every object is created from a blueprint called a **class**

```
type(42)           # <class 'int'>
type('hello')     # <class 'str'>
type([1, 2])      # <class 'list'>
```

2 The class keyword

```
class Dog:
    species = 'Canis familiaris'    # class attr
    def __init__(self, name, age):
        self.name = name           # instance attr
        self.age = age             # instance attr
    def bark(self):
        print(f'{self.name} says woof!')
```

```
rex    = Dog('Rex', 5)
pluto  = Dog('Pluto', 3)
```

```
rex.bark()    # Rex says woof!
rex.name      # 'Rex'
Dog.species   # 'Canis familiaris'
```

- ▶ `Dog(...)` **instantiates** the class, producing a new object
- ▶ Each instance gets its own name and age

3 self: the hidden first argument

```
rex = Dog('Rex', 5)
```

```
rex.bark()           # what you write
```

```
Dog.bark(rex)        # what Python actually runs
```

- ▶ `rex.bark()` is shorthand: Python passes `rex` in as `self`
- ▶ So `self` is just the instance the method was called on
- ▶ You declare `self`, but you never pass it yourself
- ▶ `__init__` works the same way and runs automatically:
`Dog('Rex', 5)` calls `__init__(rex, 'Rex', 5)`

4 Class attributes vs instance attributes

```
class Dog:
    species = 'Canis familiaris'    # shared
    def __init__(self, name):
        self.name = name           # per-instance
```

```
rex = Dog('Rex')
lola = Dog('Lola')
```

```
Dog.species    # 'Canis familiaris'
rex.species    # inherited from the class
rex.name       # 'Rex'
```

- ▶ Class attributes live on the class and are shared
- ▶ Instance attributes live on the instance and are independent

5 Instance methods

- ▶ Methods defined inside a class that operate on an instance
- ▶ `self` is always the first parameter

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f'{self.name} says woof!')

    def dog_years(self):
        return self.age * 7

buddy = Dog('Buddy', 3)
buddy.bark()           # Buddy says woof!
buddy.dog_years()      # 21
```

6 Static methods

- ▶ Methods that belong to the class but do not receive `self`
- ▶ Useful for utilities that are logically related to the class

```
class MathOps:
    @staticmethod
    def add(x, y):
        return x + y

    @staticmethod
    def square(x):
        return x * x
```

```
MathOps.add(5, 3)    # 8
```

```
MathOps.square(4)    # 16
```

```
ops = MathOps()
```

```
ops.add(2, 2)    # 4 (works on an instance too)
```

7 Magic methods

Dunder methods let objects hook into built-in syntax:

<code>__init__</code>	called on creation
<code>__str__</code>	<code>str(obj)</code> , <code>print(obj)</code>
<code>__repr__</code>	<code>repr(obj)</code> , shown in the REPL
<code>__len__</code>	<code>len(obj)</code>
<code>__add__</code>	<code>obj + other</code>
<code>__eq__</code>	<code>obj == other</code>

```
class Vec:
    def __init__(self, x):  self.x = x
    def __str__(self):     return f'Vec({self.x})'
    def __eq__(self, o):   return self.x == o.x
```

```
print(Vec(3))           # Vec(3)
Vec(3) == Vec(3)        # True
```


8 Protected and private attributes

```
class MyClass:
    def __init__(self, value):
        self._protected = value    # internal use
        self.__private  = value    # name-mangled
    def get_private(self):
        return self.__private
```

```
obj = MyClass(10)
obj._protected          # 10 ('hands off')
obj._MyClass__private   # name-mangled access
obj.get_private()       # 10 (preferred)
```

- ▶ Single underscore `_` is a convention meaning “internal”
- ▶ Double underscore `__` mangles `__x` to `_ClassName__x`
- ▶ Neither is truly hidden; Python enforces nothing

9 Properties

Properties give attribute-style access while running code:

```
class Dog:
    def __init__(self, age):
        self._age = age
    @property
    def age(self):          # getter
        return self._age
    @age.setter
    def age(self, value):  # setter
        if value < 0:
            raise ValueError('age must be >= 0')
        self._age = value
```

- ▶ `d.age` reads through the getter
- ▶ `d.age = 5` runs the setter, which validates
- ▶ `@age.deleter` likewise handles `del d.age`

10 Data classes

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
str(p)                # 'Point(x=1.0, y=2.0)'
p == Point(1.0, 2.0)  # True
```

- ▶ @dataclass writes `__init__`, `__repr__`, `__eq__`
- ▶ Fields can have defaults, e.g. `kind: str = 'dog'`
- ▶ Defaulted fields must come after the others

11 Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):           # inherits __init__
    def speak(self):
        return f'{self.name} woofs'

class Puppy(Dog):
    def __init__(self, name, owner):
        super().__init__(name)    # run parent init
        self.owner = owner
```

- ▶ A subclass inherits the parent's attributes and methods
- ▶ Override a method by redefining it in the subclass
- ▶ `super()` calls the parent's version of a method

12 Composition

A class can *contain* another object instead of inheriting:

```
class Engine:
    def start(self):
        return 'vroom'

class Car:
    def __init__(self):
        self.engine = Engine()    # composition
    def drive(self):
        return self.engine.start()

car = Car()
car.drive()    # 'vroom'
```

- ▶ Prefer composition when the relationship is “has a”, not “is a”
- ▶ A Car *has an* Engine; it is not an Engine

Further reading: `__repr__` vs `__str__`

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def __repr__(self):
        return f'Dog({self.name!r}, {self.age!r})'
    def __str__(self):
        return f'{self.name} (age {self.age})'
```

```
d = Dog('Rex', 5)
repr(d)      # dev-facing, unambiguous
str(d)       # user-facing, readable
```

- ▶ `__repr__` is shown in the REPL; prefer it if you define only one
- ▶ `print()` uses `__str__`, falling back to `__repr__` if absent

Further reading: frozen data classes and field()

```
from dataclasses import dataclass, field
```

```
@dataclass(frozen=True)
```

```
class Point:
```

```
    x: float
```

```
    y: float
```

```
p = Point(1.0, 2.0)
```

```
p.x = 3.0    # error: instance is immutable
```

```
@dataclass
```

```
class Kennel:
```

```
    dogs: list = field(default_factory=list)
```

- ▶ frozen=True makes instances immutable and hashable
- ▶ field(default_factory=list) avoids the mutable-default trap from lesson 6